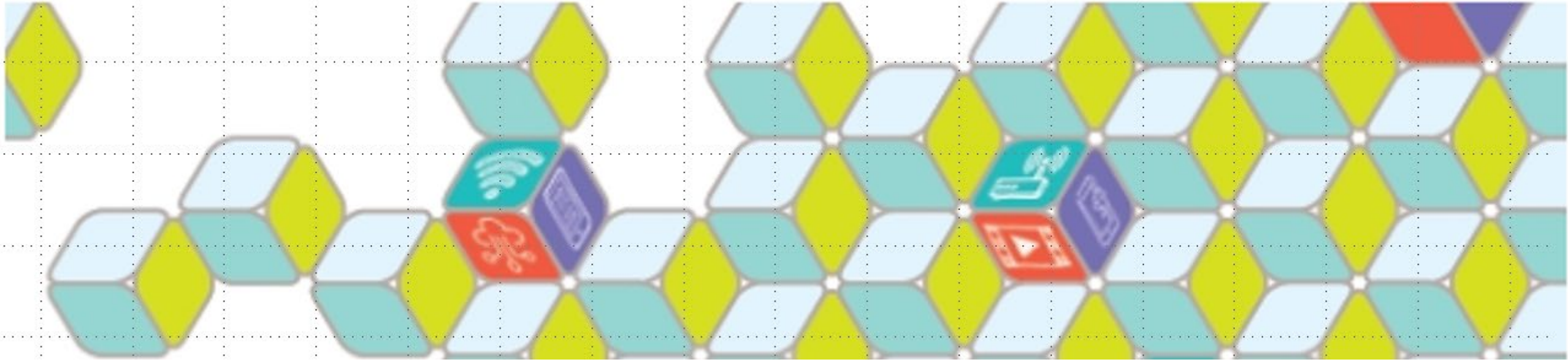


Programming Method
(using C, C++, for Games)

Prof. Tae Soo Yun
Dept. of Game, International College, Dongseo University

DSU Dongseo University 동서대학교

Ch02_Types, Operators and Expressions

The Goal of the ch.02

❖ Understanding the data types, operators and expressions Variable, constants are the basic data object (types) manipulated in a pgm.

- Operator specify what is to be done to them.
- Declarations list the variables to be used and state what types they have, what their initial values are.
- expressions combine variables and constants to produce new values.

Chapter 2

- Section 2.1 Variable Names
- Section 2.2 Data Types and Sizes
- Section 2.3 Constants
- Section 2.4 Declarations
- Section 2.5 Arithmetic Operators
- Section 2.6 Relational and Logical Operators
- Section 2.7 Type conversions
- Section 2.8 Increment and Decrement Operators
- Section 2.9 Bitwise Logical Operators
- Section 2.10 Assignment Operators and Expressions
- Section 2.11 Conditional Expressions
- Section 2.12 Precedence and Order of Evaluation

2.1 Variable Names

❖ Restrictions on variable and symbolic constant names (User defined words)

- Names are made up of letters and digits.
- The first character must be a letter.
- “_” as a letter, useful for improving the readability of long var.
- Upper and lower case are distinct. x and X are different names.
- Traditionally, lower case for var. names, All upper case for symbolic constants

- Keyword like if, else, int, float, etc., are reserved word.

2.2 Data Types and Sizes

❖ Basic data types in C

- char : a single byte, capable of holding one character in the local character set
- Int : an integer, typically reflecting the natural size of integers on the host machine
- float : single-precision floating point
- double : double-precision floating point

❖ type qualifiers (modifier)

- short and long is different according to the its own h/w
- int, normally be the natural size for a machine
- short is 16bit, long 32, int either 16 or 32bits.
- Signed or unsigned be applied to char or any integer.
- Unsigned numbers are always positive or zero, and obey the laws of arithmetic modulo 2^n, (ex. If chars 8bit, unsigned char have values 0 ~255, singed char : -128~127)

```
short int sh;
long int counter;
```

❖ grammar : [signed|unsigned] [short|long] int ⟨variable_name⟩

2.2 Data Types and Sizes

❖ Data types in Visual Studio

- According to compiler, the size of data types can be changed

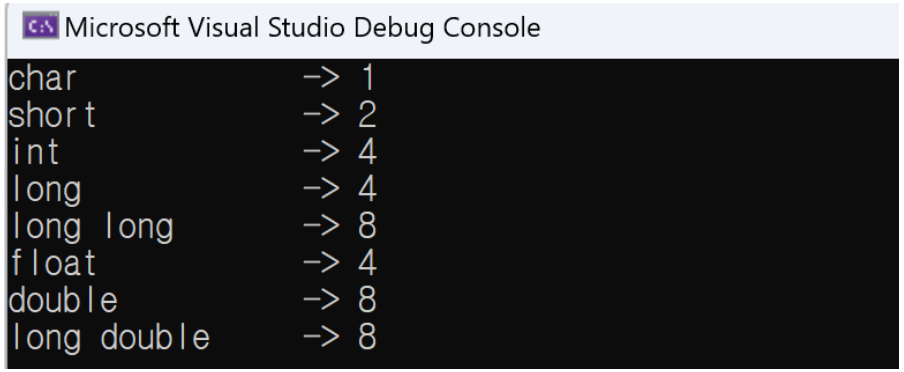
Data Types		# of Byte	Range
integer	short	2	-32768 ~ 32767
	int	4	-2147483648 ~ 2147483647
	long	4	-2147483648 ~ 2147483647
	long long	8	–9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
character	char	1	-128 ~ 127
Floating Point	float	4	1.2E-38 ~ 3.4E38
	double	8	2.2E-308 ~ 1.8E308
	long double	8	2.2E-308 ~ 1.8E308

exercise

❖ exercise 2-1.

- Write a program to determine the ranges of char, short, int, and long variables, both singed and unsigned, by printing appropriate values from standard headers and by direct computation. Harder if you compute them: determine the ranges of the various floating-point types.

- sizeof(data types) returns the size of the data types



2.3 constants

❖ variables vs. Constants

- Constants is that a data type or storage space for data that, once set, does not need to be changed
- Variables is that a data type or storage space where the stored value can be changed at any time during the execution of the program

❖Notations

- Integer constant : 1234
- Long constant : written with a terminal l or L ; 123456789L
- Unsigned constants : written with a terminal u, or U
- Float constant : f, or F
- Long double constant : l or L
- Octal, hexadecimal
  - 0 (zero) : octal ; 037
  - 0x, 0X : hexadecimal ; 0x1f, 0X1F
  - eg.)decimal 31 can be written as 037 in octal and 0x1f, 0X1F in hex

2.3 constants (continue)

❖ Notations

- Character constant is an integer, written as one character within single quotes, 'x'
  - The value of a character constant is the numeric value of the character in the machine's character set.
  - Ex) '0' has the value 48
- Can be represented string constants by escape sequences like ₩n
- Arbitrary byte-sized bit pattern can be specified by '₩ooo' or '₩xhh', where ooo is one to three digits (0…7), hh is one or more hexadecimal digits (0…9, a…f, A…F) respectively
- Using #define

```
#define VTAB '\013'     /* ASCII vertical tab */
#define BELL '\007'     /* ASCII bell character */

#define VTAB '\xb'      /* ASCII vertical tab */
#define BELL '\x7'      /* ASCII bell character */
```

- The complete set of escape sequences is

\a	alert (bell) character	\\	backslash
\b	backspace	\?	question mark
\f	formfeed	\'	single quote
\n	newline	\"	double quote
\r	carriage return	*ooo*	octal number
\t	horizontal tab	*xhh*	hexadecimal number
\v	vertical tab		

```c
#include <stdio.h>

#define VTAB '₩013'
#define BELL '₩007'

#define VTAB '₩xb'
#define BELL '₩x7'

void main()
{
    // test of  bit-pattern escape sequence
    printf("Bell sound ₩₩007 %c₩n", BELL);
    printf("VTAB ₩₩013 %c₩n", VTAB);
    printf("VTAB ₩₩'xb₩' %c₩n", VTAB);
    printf("Bell sound ₩₩'x7₩' %c₩n", BELL);

    // test of escape sequences
    printf("%c", 7);   // bell
    printf("₩a");      // bell
}
```

```
Microsoft Visual Studio Debug Console
Bell sound ₩007
VTAB ₩013

VTAB ₩'xb'

Bell sound ₩'x7'
```

2.3 constants (continue)

❖ ASCII Code (American Standard Code for Information Interchange) : https://ascii-code.com

Dec	Hex	Abbr	Code	Name	Dec	Hex	Glyph	Dec	Hex	Glyph	Dec	Hex	Glyph
0	0	NUL	\0	Null character	32	20		64	40	@	96	60	`
1	1	SOH		Start of Header	33	21	!	65	41	A	97	61	a
2	2	STX		Start of Text	34	22	"	66	42	B	98	62	b
3	3	ETX		End of Text	35	23	#	67	43	C	99	63	c
4	4	EOT		End of Transmission	36	24	$	68	44	D	100	64	d
5	5	ENQ		Enquiry	37	25	%	69	45	E	101	65	e
6	6	ACK		Acknowledgment	38	26	&	70	46	F	102	66	f
7	7	BEL	\a	Bell	39	27	'	71	47	G	103	67	g
8	8	BS	\b	Backspace	40	28	(	72	48	H	104	68	h
9	9	HT	\t	Horizontal Tab	41	29	)	73	49	I	105	69	i
10	0A	LF	\n	Line feed	42	2A	*	74	4A	J	106	6A	j
11	0B	VT	\v	Vertical Tab	43	2B	+	75	4B	K	107	6B	k
12	0C	FF	\f	Form feed	44	2C	,	76	4C	L	108	6C	l
13	0D	CR	\r	Carriage return	45	2D	-	77	4D	M	109	6D	m
14	0E	SO		Shift Out	46	2E	.	78	4E	N	110	6E	n
15	0F	SI		Shift In	47	2F	/	79	4F	O	111	6F	o
16	10	DLE		Data Link Escape	48	30	0	80	50	P	112	70	p
17	11	DC1		Device Control 1 (oft. XON)	49	31	1	81	51	Q	113	71	q
18	12	DC2		Device Control 2	50	32	2	82	52	R	114	72	r
19	13	DC3		Device Control 3 (oft. XOFF)	51	33	3	83	53	S	115	73	s
20	14	DC4		Device Control 4	52	34	4	84	54	T	116	74	t
21	15	NAK		Negative Acknowledgement	53	35	5	85	55	U	117	75	u
22	16	SYN		Synchronous idle	54	36	6	86	56	V	118	76	v
23	17	ETB		End of Transmission Block	55	37	7	87	57	W	119	77	w
24	18	CAN		Cancel	56	38	8	88	58	X	120	78	x
25	19	EM		End of Medium	57	39	9	89	59	Y	121	79	y
26	1A	SUB		Substitute	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	\e	Escape	59	3B	;	91	5B	[	123	7B	{
28	1C	FS		File Separator	60	3C	<	92	5C	\	124	7C	|

DSU

2.3 constants (continue)

❖ Using constant expression

- constant expression is an expression that involves only constants.

```
#define MAXLINE 1000
char line[MAXLINE+1];
```

or

```
#define LEAP 1   /* in leap years */
int days[31+28+LEAP+31+30+31+30+31+31+30+31+30+31];
```

❖ Using string constants, or string literal

- is a sequence of zero or more characters surrounded by double quotes, as in

```
"I am a string"
```

or

```
""   /* the empty string */
```

- String constant is an array of characters, + '₩0' (null character)
  - How much space is needed to store the string?

```
/* strlen:  return length of s */
int strlen(char s[])
{
    int i;

    i = 0;
    while (s[i] != '\0')
        ++i;
    return i;
}
```

2.3 constants (continue)

- Notations
  - 'x' vs. "x" what is the difference between two ?
  - Enumeration constant
    - Enum Boolean { NO, YES } → the first item NO has value 0, the next 1. and so on.
    - If not all values are specified, unspecified values are automatically assigned below.

```
enum escapes { BELL = '\a', BACKSPACE = '\b', TAB = '\t',
               NEWLINE = '\n', VTAB = '\v', RETURN = '\r' };

enum months { JAN = 1, FEB, MAR, APR, MAY, JUN,
              JUL, AUG, SEP, OCT, NOV, DEC };
                    /* FEB is 2, MAR is 3, etc. */
```

2.4 Declarations

❖ all variables must be declared before use, usually at the beginning of the function before any executable statements.

```
int  lower, upper, step;
char c, line[1000];
```

```
int  lower;
int  upper;
int  step;
char c;
char line[1000];
```

❖ a variable may be initialized in its declaration.

```
char  esc = '\\';
int   i = 0;
int   limit = MAXLINE+1;
float eps = 1.0e-5;
```

❖ qualifier const : used to not change the value.

```
const double e = 2.71828182845905;
const char msg[] = "warning: ";
```

❖Can be used with array arguments, so that the function does not change that array :

```
int strlen(const char[]);
```

2.5 ~ 2.10 Operators

❖ Arithmetic Operators

- Binary arithmetic operators are +, −, *, /, and the modulus operator %.
- Ex) A problem to determine whether a given year is a leap year
  - ✓ Leap year : divisible by 4, not by 100, except divisible by 400

```
if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
    printf("%d is a leap year\n", year);
else
    printf("%d is not a leap year\n", year);
```

- Precedence among the operators.

❖ Relational and Logical Operators

- 〉, 〉−, 〈, 〈=
- ==, !=
- Relational op. hav lower precedence that arithm. Op.
- So, i 〈 lim −1 is taken as i 〈 (lim−1)

TABLE 2-1. PRECEDENCE AND ASSOCIATIVITY OF OPERATORS

OPERATORS	ASSOCIATIVITY
`()  []  ->  .`	left to right
`! ~ ++ -- + - * & (type) sizeof`	right to left
`*  /  %`	left to right
`+  -`	left to right
`<<  >>`	left to right
`<  <=  >  >=`	left to right
`==  !=`	left to right
`&`	left to right
`^`	left to right
`|`	left to right
`&&`	left to right
`||`	left to right
`?:`	right to left
`= += -= *= /= %= &= ^= |= <<= >>=`	right to left
`,`	left to right

Unary +, −, and * have higher precedence than the binary forms.

2.5 ~ 2.10 Operators (continue)

❖ Type conversions

- Two type of type conversion : implicit, explicit
- Implicit type conversion

If either operand is `long double`, convert the other to `long double`.

Otherwise, if either operand is `double`, convert the other to `double`.

Otherwise, if either operand is `float`, convert the other to `float`.

Otherwise, convert `char` and `short` to `int`.

Then, if either operand is `long`, convert the other to `long`.

- Explicit type conversion
  - (type name) expression : ex) sqrt((double) n)

- Ref)

```
Given a function prototype for sqrt
double sqrt(double);
The call
root2 = sqrt(2);
coerces the integer 2 into the double
value 2.0
```

```
unsigned long int next = 1;

/* rand:  return pseudo-random integer on 0..32767 *
int rand(void)
{
    next = next * 1103515245 + 12345;
    return (unsigned int)(next/65536) % 32768;
}

/* srand:  set seed for rand() */
void srand(unsigned int seed)
{
    next = seed;
}
```

Exercise 2-3

❖ write the function htoi(s), which converts a string of hexa-decimal digits (including an optional 0x or 0X) into its equivalent integer value. The allowable digits are 0 through 9, an through f, and A through F.

Edit code below:

```
1  int atoi(s) /* convert s to integer */
2  char s[];
3  {
4      int i, n;
5
6      n = 0;
7      for (i = 0; s[i] >= '0' && s[i] <= '9'; ++i)
8          n = 10 * n + s[i] - '0';
9      return(n);
10 }
```

Expected output from your program:

```
htoi('84ada') = 543450
htoi('f') = 15
htoi('F0') = 240
htoi('12fab') = 77739
```

The main program which will execute your code:

```
#include <stdio.h>
int main() {
  int htoi();
  printf("htoi('84ada') = %d\n", htoi("84ada"));
  printf("htoi('f') = %d\n", htoi("f"));
  printf("htoi('F0') = %d\n", htoi("F0"));
  printf("htoi('12fab') = %d\n", htoi("12fab"));
}
```

2.5 ~ 2.10 Operators (continue)

❖ Increment and Decrement Operators

- ++, --
- The increment operator ++ adds 1 to its operand, while the decrement op. - subtracts 1.
- Be careful) n =5, then x = n++; vs. x = ++n;
- inc., dec. operators can only be applied to var. (i+j)++ is illegal
- Function squeeze(s,c) which removes all occurrences of the character c from the string s

```
/* strcat:  concatenate t to end of s; s must be big enough */
void strcat(char s[], char t[])
{
    int i, j;

    i = j = 0;
    while (s[i] != '\0')    /* find end of s */
        i++;
    while ((s[i++] = t[j++]) != '\0')   /* copy t */
        ;
}
```

```
/* squeeze:  delete all c from s */
void squeeze(char s[], int c)
{
    int i, j;

    for (i = j = 0; s[i] != '\0'; i++)
        if (s[i] != c)
            s[j++] = s[i];
    s[j] = '\0';
}
```

```
if (s[i] != c) {
    s[j] = s[i];
    j++;
}
```

```
if (c == '\n') {
    s[i] = c;
    ++i;
}
```

```
if (c == '\n')
    s[i++] = c;
```

Exercise 2-4,5

❖Ex 2-4

- Write an alternate version of squeeze(s1,s2) that deletes each character in s1 that matches any character in the string s2.

❖Ex 2-5

- Write the function any(s1,s2), which returns the first location in the string s1 where any character from the string s2 occurs, or -1 if s1 contains no character from s2. (The standard lib. Function strpbrk does the sam job but returns a pointer to the location)

2.5 ~ 2.10 Operators (continue)

❖ Bitwise Operators

- Provides six operators for bit manipulation.
- AND op. & is used to mask off some set of bits
  - ✓ n = n & 0177; set to zero all but the low-order 7 bits of n
  - ✓ n = n & 001111111
- OR op. | is used to turn bits on :
  - ✓ x = x | SET_ON;
- Be careful) bitwise op. & and |, logical op. && and ||
  - ✓ If x=1,y=2, x&y = 0, x&&y = 1

✓ Shift op ⟨⟨ and ⟩⟩, perform left and right shifts of value by the #of bit

- ✓ X ⟨⟨ 2 shifts the value of x left by two pos. Equivalent to multiplication by 4.

- Unary op. ~ yields the one's complement of an integer.
  - ✓ X = x & ~077 sets the last six bit of x to zero

&	bitwise AND
|	bitwise inclusive OR
^	bitwise exclusive OR
<<	left shift
>>	right shift
~	one's complement (unary)

2.6~8, Exercise

❖ exercise 2.6

- Write a function setbits(x,p,n,y) that returns x with the n bits that begin at position p set to the right most n bits of y, leaving the other bits unchanged.

❖Exercise 2.7

- Write a function invert(x,p,n) that returns x with the n bits that begin at position p inverted (i.e., 1 changed into 0 and vice versa), leaving the others unchanged.

❖ exercise 2.8

- Write a function rightrot(x,n) that returns the value of the integet x rotated to the right by n bit positions.